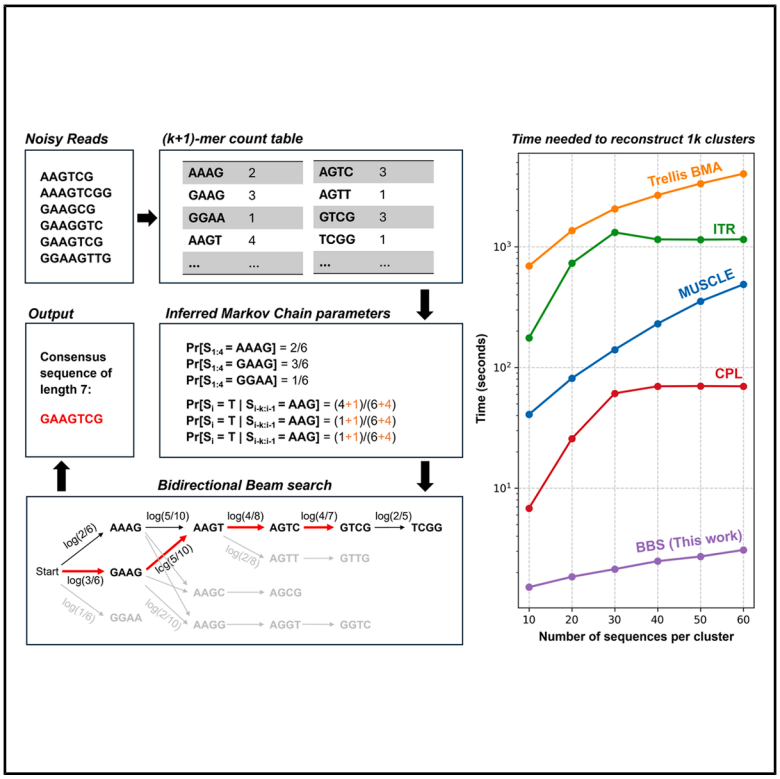
iScience

Article

Efficient trace reconstruction in DNA storage systems using bidirectional beam search

Graphical abstract



Highlights

- BBS is a linear-time consensus finding algorithm for DNA data storage
- BBS works without training or making strict assumptions on error rates
- BBS is ~20x faster than the other algorithms, while achieving top-tier accuracy
- Confidence scores reported by BBS can help build a reliable DNA storage system

Authors

Zhenhao Gu, Hongyi Xin, Puru Sharma, Gary Yipeng Goh, Limsoon Wong, Niranjan Nagarajan

Correspondence

wongls@comp.nus.edu.sg (L.W.), nagarajann@a-star.edu.sg (N.N.)

In brief

Biocomputational method; Data storage representation; Signal reconstruction; Computing methodology

Gu et al., 2025, iScience *28*, 113791
November 21, 2025
https://doi.org/10.1016/j.isci.2025.113791

CellPress

iScience

CellPress
OPEN ACCESS

Article

Efficient trace reconstruction in DNA storage systems using bidirectional beam search

Zhenhao Gu,[1,2] Hongyi Xin,[3] Puru Sharma,[1] Gary Yipeng Goh,[1] Limsoon Wong,[1,*] and Niranjan Nagarajan[1,2,4,5,*]
[1]Department of Computer Science, School of Computing, National University of Singapore, Singapore 117417, Singapore
[2]Genome Institute of Singapore, A*STAR, Singapore 138672, Singapore
[3]Global Institute of Future Technology, Shanghai Jiao Tong University, Shanghai 200240, China
[4]Yong Loo Lin School of Medicine, National University of Singapore, Singapore 117596, Singapore
[5]Lead contact
*Correspondence: wongls@comp.nus.edu.sg (L.W.), nagarajann@a-star.edu.sg (N.N.)
https://doi.org/10.1016/j.isci.2025.113791

SUMMARY

As DNA data storage gains popularity, efficient trace reconstruction algorithms are crucial for fast decoding of data from noisy sequenced reads (or "traces"). Existing approaches, often adaptations of multiple sequence alignment or read correction methods, rely on strict assumptions of fixed error rates, showing limited generalizability to more complex datasets and with slower running times. We introduce a probabilistic formulation of the trace reconstruction problem by modeling traces as observations from a *k*-th order Markov chain. Instead of doing alignment, we identify the sequence most likely generated by the Markov chain as the consensus. This inspires bidirectional beam search (BBS), an algorithm that reconstructs the consensus in linear time with respect to its length. Experiments on multiple public Nanopore sequencing datasets demonstrate that BBS achieves top-tier accuracy while being approximately 20× faster than existing methods, showing its potential to enhance the efficiency and reliability of DNA data storage systems.

INTRODUCTION

DNA data storage has emerged as a transformative solution for modern information storage thanks to its extraordinarily high data density (up to 455 billion GB per gram),[1] long-term data integrity, and low energy consumption.[2,3] In this paradigm, digital data are encoded as nucleotide sequences, synthesized into DNA strands, and later retrieved through PCR and sequencing.[4] The trace reconstruction problem, which aims to reconstruct the original data from the error-prone sequenced reads, or "traces", is vital in building an efficient and reliable DNA data storage system.[5] An ideal reconstruction algorithm must be fast to reduce latency, accurate to prevent data loss or re-sequencing, and be able to work with as few traces as possible to eliminate the need for multiple PCR rounds and deeper sequencing coverages.

Recent advances in third-generation sequencing technologies, notably Oxford Nanopore's portable sequencers, have expanded the capabilities of DNA data storage.[6,7] By supporting real-time sequencing of longer read lengths, higher sequencing speeds (<0.02 s per base per pore,[8] compared to >2 min per base in Illumina sequencing[9,10]), and PCR-free protocols,[11] Nanopore technology offers the potential to increase data density and reduce read latency significantly. However, these benefits are offset by a higher mean error rate of around 6%,[12] which, along with errors from cost-effective synthesis methods,[13] greatly complicate the reconstruction process. Additionally, unlike genome assembly and multiple sequence alignment (MSA) problems, DNA storage systems can leverage prior knowledge such as the length of the encoded sequence and error correction codes (ECC) to aid in reconstruction.[14] Effectively integrating this prior knowledge while mitigating errors in the reads presents significant challenges to the trace reconstruction algorithms.

Many algorithms have been proposed to solve the trace reconstruction problem. Based on the various optimization goals, the methods can generally be categorized into alignment-based, IDS-channel-based, assembly based, and deep-learning-based.

Several popular algorithms used for MSA have also been used for the trace reconstruction problem. For example, Antkowiak et al. utilized the alignment results of MUSCLE along with a weighted majority voting to find the consensus sequence,[13,15] and Xie et al. demonstrated superior error-correcting capabilities using MAFFT.[16,17] MSA algorithms have also been shown to have good performance under high error rates.[13] However, finding global optimal scoring metrics for the alignment under different synthesis and sequencing conditions is not trivial and has to be determined empirically (Figure 1). In fact, it has been demonstrated that the error situation is highly contextual in Nanopore sequencing and a global error scoring mechanism is often inadequate at encapsulating the nuanced error patterns in Nanopore reads.[18] Moreover, due to the exponential time complexity as a function of the number of sequences, MSA algorithms are limited to small clusters of reads for efficient decoding and cannot handle elevated sequencing depths.

Insertion-deletion-substitution (IDS)-channel-based methods model the DNA synthesis and sequencing process as a

communication channel, where insertions, deletions, and substitutions occur at each sequence position with probabilities p_I, p_D, and p_S, respectively.[19] These methods aim to reconstruct the original sequence by finding the consensus string that maximizes the likelihood of observing the given traces[20] under the aforementioned assumption. In practice, this optimization is typically performed by iteratively correcting errors in the reads until they converge to a consensus, as seen in the bitwise majority alignment (BMA) algorithm.[21] Several variants of BMA have been proposed, including BMA lookahead, which improves error correction by looking at a longer range of bases in the reads,[22–24] and Trellis BMA, which refines the IDS-channel with a hidden Markov model.[19] Although these approaches are theoretically well-founded under the IDS-channel model, they may face challenges when applied to real datasets, where the assumption of independent error occurrence does not always hold. This is especially evident for third-generation sequencing technologies, such as Nanopore, where base-pairs are not individually sequenced but rather DNA fragments are indirectly deduced by decoding the electrochemical signals of consecutive overlapping 10-to-20 base-pair oligo fragments. Consequently, successive errors are not independent and sequencing errors are often more prevalent at the two ends of a read than the middle section,[13,25] leading to deviations from the expected error model. Similar to MSA algorithms, building a universally robust error model remains a challenge.

While previous methods depend largely on the chosen hyperparameters and/or the error model determined using the training data, the assembly based methods offer a ''parameter-free'' approach, aiming to maximize the number of reads that agree with the subsequences of the final consensus. This is achieved by greedily identifying the maximum-weighted path in the de Bruijn graph using DBGPS[26] and by assembling the longest common subsequences between each pair of reads in the iterative algorithm (ITR).[27] These methods demonstrated superior accuracy across different datasets. However, they are computationally intensive and fail to fully account for the prior knowledge of the encoded sequence.

Recently, deep-learning-based methods emerged as a popular alternative for trace reconstruction, which optimizes the alignment and consensus construction process by auto-learning the error patterns. RobuSeqNet assigns weights to each read using convolutional layers and an attention mechanism,[28] and finds the consensus sequence based on the weighted sum of all sequences within the cluster. DNA-GAN[29] and single-read reconstruction (SRR) algorithm[30] attempted to output the consensus sequence directly in the network output. Such methods show promising results but incur high computational costs and require special hardware. Furthermore, they might not perform equally well given unseen error distributions.

In this work, we aim to propose a computationally lightweight yet highly accurate trace reconstruction algorithm that is capable of reproducing the original data sequence at low sequencing depths. Our work extends both assembly based and deep-learning-based methods for trace reconstruction algorithms. Rather than optimizing sequence alignment or learning a global error model for the entire dataset, we model the traces within each cluster as observations from a k-th order Markov chain and aim to predict the sequence with the highest likelihood of being emitted (Figure 2). This approach allows error probabilities to be estimated independently at each position within every strand cluster, providing greater flexibility in handling complex sequencing errors.

To efficiently identify the most probable sequence, we introduce the bidirectional beam search (BBS) algorithm, which leverages the learned Markov chain to determine the most likely next trace. Notably, the computational complexity of the reconstruction phase of our BBS algorithm scales linearly with the length of the consensus sequence, making it highly efficient. Experimental results on multiple Nanopore datasets and various cluster sizes demonstrate that our approach is among the most accurate methods when compared with the state-of-the-art algorithms while being approximately 20× faster, highlighting its potential to improve the efficiency of DNA data storage pipelines significantly.

RESULTS

To test the efficiency and correctness of our algorithm, we compare the performance against the state-of-the-art algorithms in each type of method for trace reconstruction. In particular, MUSCLE[15] from the MSA-based methods, TrellisBMA[19] from the IDS-channel-based methods, and ITR[27] from the assembly like methods. We utilized the latest MUSCLE v5[31] and implementation by Antkowiak et al. to return the consensus sequence from the alignment. The newest deep-learning-based methods were not included due to the lack of publicly available code.[29] Instead, we compare our results with their reported accuracy.

Additionally, we compare our method against a newly published method, conditional probability logic (CPL), which is a refinement step for the deep learning method DNAFormer.[32] Though BBS was developed independently of CPL, they share a similar methodology. CPL utilizes a first-order Markov chain with the conditional probabilities estimated using the pairwise alignments, instead of k-mer counting in BBS. As a result, CPL uses quadratic time with respect to the number of traces in each cluster. In addition, CPL finds the optimal longest path in the constructed graph, whereas BBS employs a greedy algorithm for efficiency.

All experiments are run on a machine with an Intel Core i9-13900H CPU, with 20 threads and 16 GB of memory. MUSCLE is the only tool with a multi-threaded implementation. All tools are run using their default parameters. For BBS, the beam width is chosen to be 20. No error correction code is assumed.

BBS allows accurate and efficient trace reconstruction from real nanopore reads

We test the five algorithms on real datasets, with three open-source datasets published by Bar-Lev et al.[32] (Nanopore two flowcells), Srinivasavaradhan et al.,[19] and Chandak et al.,[6] as shown in Table 1. The dataset from Bar-Lev et al. is randomly down sampled to 10,000 clusters so that the tested tools finish in a reasonable run time. Clustering for the dataset from Chandak et al. is done by simply mapping the sequenced reads to

Table 1. Datasets used for evaluation of the algorithms

Dataset	Bar-Lev et al.[32]	Srinivasavaradhan et al.[19]	Chandak et al.[6]
#Clusters	10000	9984	1466
Synthesis tech	Twist Bioscience	Twist Bioscience	CustomArray
Sequencing tech	MinION	MinION	MinION
Clustering alg.	Bar-Lev et al.[32]	Rashtchian et al.[14]	Perfect
L	140	110	108
Coverage	21.37	27.01	114.29
p_D	1.17%	1.86%	5.09%
p_I	1.52%	2.14%	4.56%
p_S	1.65%	1.77%	3.91%
Error rate	4.34%	5.77%	13.56%

Statistics of the three real datasets. The error rates are estimated using the script by Srinivasavaradhan et al.[19].

the closest sequence in the ground truth (perfect clustering). On the other hand, the dataset Bar-Lev et al. uses its own binning algorithm,[32] and Srinivasavaradhan et al. utilize the clustering algorithm from Rashtchian et al.[14] The performance of each algorithm is listed in Table 2.

BBS consistently achieves top-tier reconstruction accuracy across all three datasets while significantly reducing the running time. Against MUSCLE, Trellis BMA, and ITR, BBS achieves 3–6x smaller Hamming distance, indicating its ability to reliably reconstruct the encoded sequence, especially for datasets with high error rates and higher coverage. In addition, BBS also demonstrates the best performance in the Nanopore dataset in Sabary et al.[27] (Table S2), showing its ability to generalize across various datasets.

Notably, the success rate of the BBS algorithm in the dataset from Srinivasavaradhan et al. (94.772%) also surpasses the reported success rate of deep learning methods, such as 85.42% in DNAFormer[32] and 91.73% in DNA-GAN,[29] without the need for training and post-processing.

In the meantime, with the same computational resources, BBS is also ∼20x faster than the state-of-the-art algorithms, using only seconds for datasets that used to take minutes to hours to reconstruct. To test the time complexity of the algorithms, we generated clusters of size ranging from 10 to 60 to test the time complexity of the algorithms (Table S1). Both BBS and Trellis BMA scale linearly with the cluster size. However, Trellis BMA shows a much larger constant factor, making it the slowest of the tested algorithms. MUSCLE, being a MSA algorithm, demonstrated an exponential increase with respect to the number of traces. Finally, both ITR and CPL showed a quadratic increase for cluster sizes 10–30 while being roughly constant for larger cluster sizes. This is because ITR reduces the runtime by using only the first 25 traces for cluster sizes larger than 25.[27] Such a strategy works fine for small error rates or datasets with low coverage but demonstrates a decrease in performance for datasets with high coverage, such as the Chandak dataset, due to the loss of information.

In addition, we plot the probability that the ground truth sequence s disagrees with the reconstructed sequence $\hat{s}$ on the i-th position, $\Pr[s_i \neq \hat{s}_i]$, against i in Figure 3. Both MUSCLE and ITR show a gradual increase in error rate, which is expected because an early wrong decision or an insertion or deletion error at the front of the reconstructed sequence can lead to disagreement in all subsequent positions. Both methods also show a sudden increase in error rate toward the end of the sequence due to the fact that the prior information of the encoded sequence length L is not fully utilized. As a result, the reconstructed sequence of these two methods

Table 2. Accuracy and running time of the algorithms on the public datasets

Dataset	Tools	Success rate	Avg. edit distance	Avg. Hamming distance	Running time (s)
Bar-Lev et al.[32]	MUSCLE	96.25%	0.258	1.382	1475.55
	Trellis BMA	92.41%	0.363	1.635	19759.85
	ITR	97.42%	0.221	0.975	12462.04
	CPL	*98.39%*	**0.201**	*0.374*	*495.22*
	BBS	**98.80%**	*0.206*	**0.346**	**23.84**
Srinivasavaradhan et al.[19]	MUSCLE	84.70%	0.259	6.032	1741.65
	Trellis BMA	69.57%	0.908	6.003	17859.17
	ITR	87.58%	0.232	4.797	7351.52
	CPL	**94.93%**	**0.150**	**1.286**	*361.05*
	BBS	*94.77%*	*0.168*	*1.616*	**20.01**
Chandak et al.[6]	MUSCLE	76.94%	0.366	8.821	7082.75
	Trellis BMA	52.32%	1.608	13.094	10744.21
	ITR	64.12%	0.666	12.231	1614.64
	CPL	*90.93%*	**0.205**	**2.299**	*131.60*
	BBS	**91.34%**	*0.283*	*2.738*	**8.52**

Performance of MUSCLE,[13,15] Trellis BMA,[19] ITR,[27] CPL,[32] and BBS (this work) on the three real datasets. The bold text indicates the best-performing tool, while the italic text indicates the second-best one.

Alignment A:

```
seq1: AGTCC-ACT
seq2: AGAAC--TT
seq3: AGTCCCATT
cons: AGTCC-ATT
```

Alignment B:

```
seq1: AGTCC-ACT-
seq2: AGA---ACTT
seq3: AGTCCCA-TT
cons: AGTCC-ACTT
```

Figure 1. An example showing why multiple sequence alignment may fail

The optimal alignment and consensus (denoted `cons` in the figure) may differ given different scoring metrics. Neither of the alignments is correct if given the prior knowledge that the length of the encoded sequence is 10, in which case the optimal consensus would be `AGTCCCACTT`.

often varies in length. On the other hand, Trellis BMA exhibits a triangular shape since its reconstruction process starts from the two ends and joins in the middle of the sequence. It also shows a larger slope compared with MUSCLE and ITR.

Different from the other methods, the error curves of BBS and CPL are the closest to uniform. BBS chooses the best paths out of all the length-l paths, leveraging the prior information on the length of the encoded sequence, and maximizing the global joint likelihood of observing the whole sequence, rather than focusing on local subsequences.

While BBS has among the best error correction performance in real data, its advantage over the other algorithms significantly reduces in synthetic data where errors are assumed to be independent and uniformly random (Figure S1). The discrepancy of error correction performance between real and synthetic datasets highlights the limitation of over-simplifying the Nanopore error model, and demonstrates the advantage of the k-MC approach of BBS.

BBS achieves top-tier accuracy at all sequencing coverages

Scoring a higher error correction percentage is essential to reducing cost and latency for data retrieval in DNA storage systems. When a trace reconstruction fails, the entire lengthy biochemical process has to be repeated afresh through an expensive re-read. While the accuracy of a trace reconstruction algorithm can be improved by having larger clusters, it involves ramping up the PCR amplification and increasing the sequencing depth, both of which increase the latency and the cost of a data read. Thus, achieving high reconstruction accuracy at shallow sequencing depth is instrumental in reducing the re-read frequency and reducing stuttering in data retrieval while maintaining a low-cost DNA storage system.

To evaluate the performance of our reconstruction algorithm at varying sequencing depths, we randomly sampled 500 clusters from the Srinivasavaradhan et al. dataset[19] and performed a subsampling experiment. For each cluster, we randomly selected a subset of reads to maintain a fixed cluster size. We began by subsampling each cluster to a size of exactly 2, then repeated the process for cluster sizes ranging from 4 to 30. The failure rate, defined as 1-success rate, was plotted against the cluster size (coverage) in Figure 4.

As expected, all algorithms show improved performance with larger coverage. BBS and CPL consistently outperform the others across all cluster sizes. They also demonstrate the fastest decrease in failure rate, converging to the lowest failure rate. Notably, BBS achieves a success rate of at least 95% at coverage of just 12 reads, while the MSA- and IDS-channel-based algorithms only reach this threshold at a

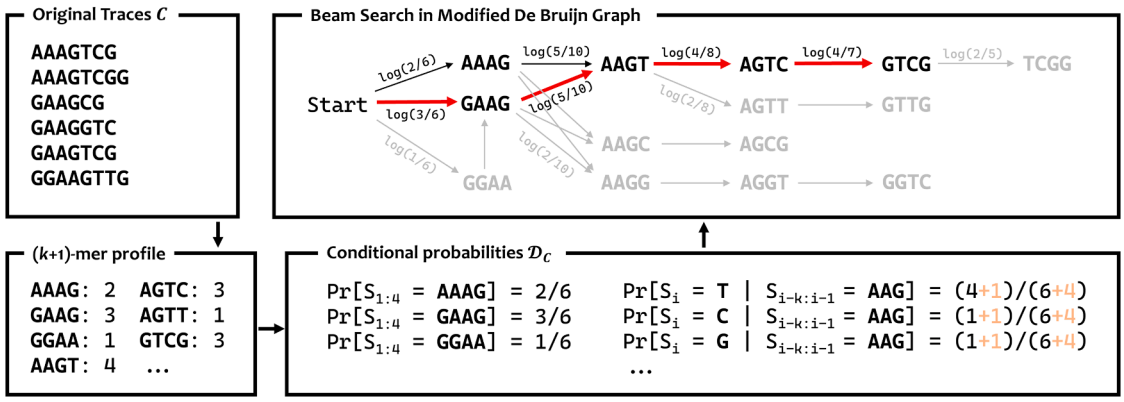


Figure 2. Workflow of the BBS algorithm

The general workflow of the BBS algorithm. In the first step, the set of original traces C (top left) is converted into a hash table that maps each (k+1)-mer to the number of times it appears in C (bottom left). k is chosen such that all the k-mer appear at most once in each trace. We then use the (k+1)-mer profile to estimate the conditional probabilities $\mathcal{D}_C$ (bottom right), which replaces the edge weights in the De Bruijn graph (top right). Finally, beam search (with beam width $B = 2$ in the figure) is performed to find the best w paths of length L in the De Bruijn graph. It saves time by skipping paths that are not promising (marked gray) in the graph. For clarity, only one directional beam search is shown in the figure.

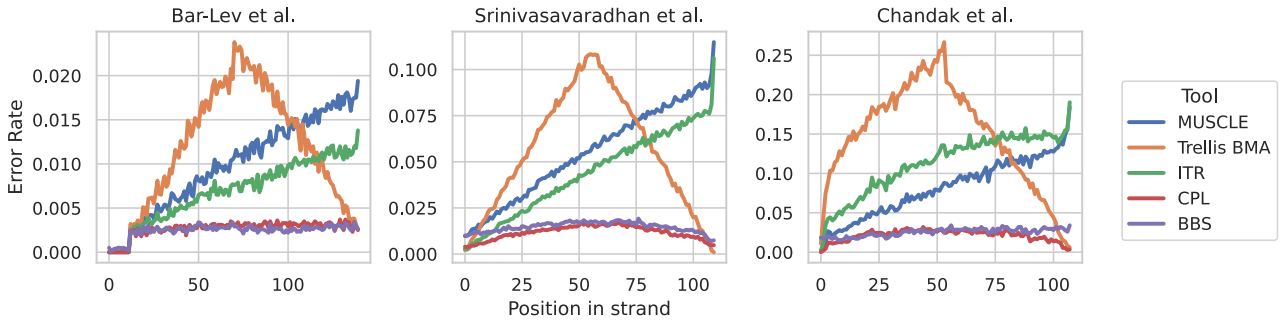


Figure 3. Error rates of the algorithm at different positions in the template

Probability of the ground truth sequence disagreeing with the reconstructed sequence on the i-th position, for all $1 \leq i \leq L$.

coverage of 30 or higher, striking a 60% sequencing cost reduction for high-success reads. This highlights the ability of our algorithm to perform effectively with smaller coverages, offering the potential to shorten latency and save costs by reducing the required sequencing depth or PCR amplification rounds.

The path weight returned by BBS is a good indicator of reconstruction quality

Previous non-deep-learning-based trace reconstruction methods only output one reconstructed sequence per cluster with no indicator of reconstruction quality. However, with the k-MC model and beam search, we are able to output multiple candidate reconstructions, each associated with a path weight that represents the log likelihood of observing the sequence as the output of the k-MC model. Furthermore, we can also calculate a confidence score for the returned sequence. Let w_i be the weight of the i-th path (denoted p_i) in the final frontier, the confidence for the i-th path can be calculated using a softmax function,

$$\text{confidence}(p_i) = \frac{\exp(w_i)}{\sum_{j=1}^{B} \exp(w_j)},$$

which intuitively represents the probability of the i-th path being the output of the k-MC, given that the output is one of $p_1, \ldots, p_B$.

We examine the distribution of path weights and the confidence score of correctly and incorrectly reconstructed clusters from the dataset from Srinivasavaradhan et al., respectively (Figure 5). It is clear that the correct and incorrect reconstructions result in a very different distribution of path weights and confidence scores. In particular, the incorrect reconstructions tend to have lower path weights and lower confidence scores.

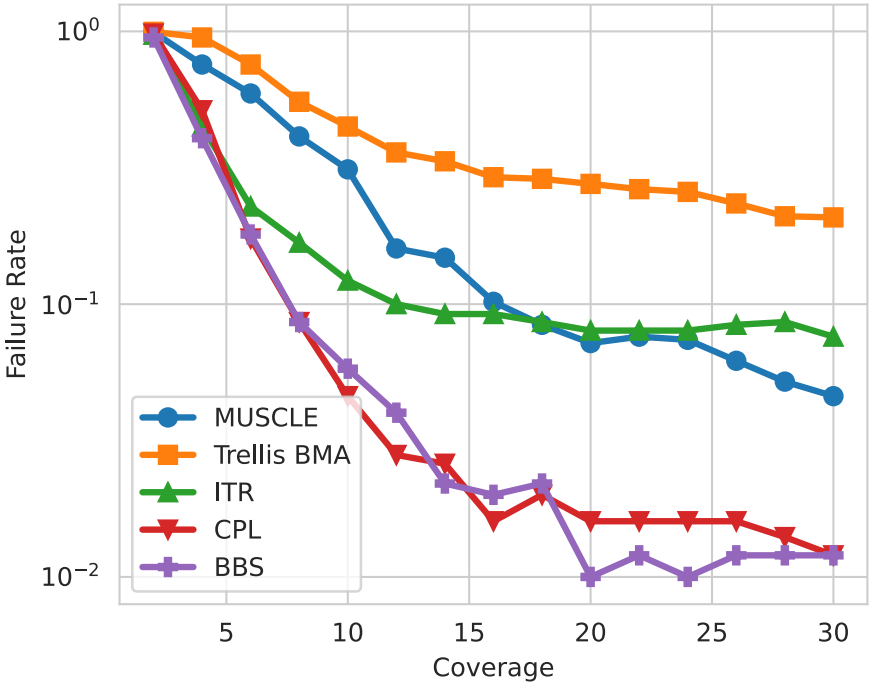


Figure 4. Performance of the algorithms with respect to cluster size

Failure rate of the algorithms at small coverages in the dataset from Srinivasavaradhan et al.[19]

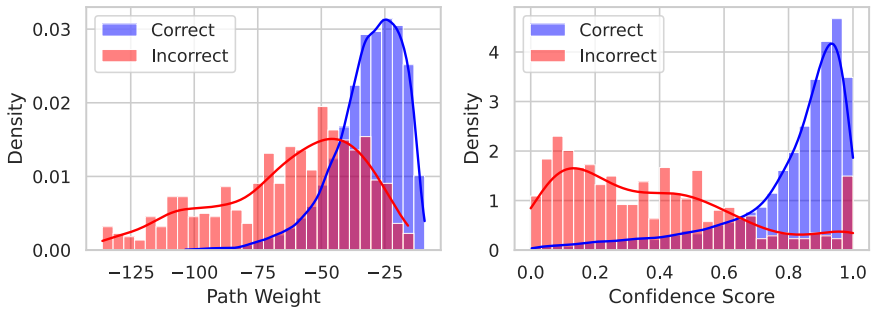


Figure 5. Confidence score reflects the quality of the reconstruction

Distribution of the path weight and confidence score for the correctly (blue) and incorrectly (red) reconstructed clusters in the dataset from Srinivasavaradhan et al.[19]

The areas under the receiver operating characteristic curve (AUROC) of the path weight and the confidence score are 0.85 and 0.87, respectively.

We also attempt to fit a simple logistic regression model using the path weight and confidence score to predict whether the reconstruction is correct or not. On the dataset from Srinivasavaradhan et al., the logistic regression model achieves an AUROC of 0.94. This result indicates the potential use of returned path weights by the BBS algorithm to infer the correctness of reconstruction and a hybrid algorithm where more candidate reconstructions from the beam search or more sophisticated but slower algorithms can be used for clusters with low confidence scores, further enhancing the accuracy of the reconstruction.

Each component of BBS is essential for high accuracy

Finally, we conduct an ablation study to test the performance of BBS with each component removed. Specifically, we apply the same beam search algorithm to a graph where the edge weights are defined by standard de Bruijn graph (DBG) edge weights based on k-mer counts. The resulting performance is significantly lower than BBS and only marginally higher than the ITR algorithm ("DBG weight" in Table 3). In addition, the algorithm's performance also decreases significantly if the Laplace smoothing is not applied ("No smoothing" in Table 3). This shows that our replacement of edge weights with the log of conditional probabilities with Laplace smoothing is well-suited for the task of consensus finding.

We also highlight the superiority of BBS by checking the performance of the algorithm with a one-directional beam search ("Unidirectional" in Table 3) and a beam width of $B = 1$ ("Greedy Search" in Table 3). The consistent superior performance of BBS over one-directional beam search across all beam widths (Figure S2) further supports the effectiveness of path weight as an indicator of reconstruction correctness. Additionally, the greedy best-first search exhibited a significant drop in success rate, highlighting its reduced flexibility and accuracy compared to the beam search.

Table 3. Ablation studies

Dataset	Bar-Lev et al.[32]	Srinivasavaradhan et al.[19]	Chandak et al.[6]
BBS	**98.80%**	**94.77%**	**91.34%**
DBG weight	−0.39%	−4.85%	−8.33%
No smoothing	−1.90%	−2.75%	−11.74%
Unidirectional	−0.13%	−0.72%	−0.82%
Greedy Search	−2.50%	−11.01%	−14.33%

Ablation study results showing the change in success rate on the three datasets when components of BBS are removed, compared with the success rate of the full BBS algorithm (in bold).

DISCUSSION

The recent surge in the popularity of DNA data storage has prompted the emergence of many algorithmic problems. The trace reconstruction problem, being the core step in the decoding process, is crucial for an accurate, fast, and reliable DNA storage system. While previous methods focused on optimizing the seed string to maximize the likelihood of observing the traces, we propose an objective to predict the sequence that is most likely the next trace in the cluster. The likelihood can be calculated by modeling the traces as observations of a k-th order Markov chain, whose parameters can be estimated simply by counting the $(k+1)$-mers in the cluster.

This problem formulation and model inspire an alignment-free algorithm named BBS, which predicts the most likely next trace by replacing weights in the De Bruijn graph with the modeled probabilities in the k-MC and finding the longest path with a fixed length with beam search. The beam search is performed once from the start of the sequences, and once from the end. The beam search operates in linear time with respect to the length of the encoded sequence.

Experiments on real datasets sequenced by Nanopore show that BBS is uniquely accurate and fast, especially when dealing with datasets with high error rates and large coverage. A simulated study also demonstrated that BBS performs the best for all coverages, achieving the same success rate using fewer traces than the other algorithms. Moreover, unlike the previous methods that only report the reconstructed sequence without any indicator of the quality of reconstruction, BBS reports the path weight along with a confidence score, which proves to be a reliable way to find the wrongly reconstructed sequences, aiding future post-processing and decoding. The experiment results show the potential of the BBS algorithm to greatly enhance the efficiency of the current DNA data storage pipeline.

Limitations of the study

Possible future directions to improve the algorithms involve optimizing the choice of parameters, including the beam width

Algorithm 1. Beam Search for Trace Reconstruction

Input: Learned probabilities from the traces $\mathcal{D}_C$, Beam width B, Length of sequence L, Length of k-mer k, Target vertex v.
Output: Best Length-L path in the De Bruijn graph.

Function BeamSearch($\mathcal{D}_C$, B, L, k, v)
  /* Initialization of the frontier */
  Frontier ←[];
  foreach (k+1)-mer u at the front of traces in C **do**
    Insert $([u], \log \Pr_{S\sim\mathcal{D}_C}[S_{1:k+1} = u])$ into Frontier;
  end
  Frontier ← top B tuples by weights in Frontier;
  /* Run L–k iterations of beam search */
  for *iteration* ←1 to L–k **do**
    NewFrontier ← [];
    foreach (*path*,w) in *Frontier* **do**
      u←path[-1]; // Last k-mer in the path
      foreach $j\in\{A,C,G,T\}$ **do**
        $u'\leftarrow(u_2,u_3,\ldots,u_{k+1},j)$;
        if u' *appeared* in the *traces* **then**
          NewPath ← path appended with u';
          $w' \leftarrow w + \log \Pr_{S\sim\mathcal{D}_C}[S_i = j | S_{i-k:i-1} = u_{1:k}]$;
          Insert (NewPath,w') into NewFrontier;
        end
      end
    end
    NewFrontier ← top B tuples by weights in NewFrontier;
    if *NewFrontier is empty* ***then***
    break;
    // There are no candidate paths longer than the ones in the current frontier
    end
    Frontier ←NewFrontier;
  end
  /* Find the highest weight path, preferably ending with v */
  CandidatePaths ← {(path,w) ∈ Frontier|path[-1] = v};
  if *CandidatePaths is not empty* **then**
    return (path, weight) with the highest weight from CandidatePaths;
  else
    return (path, weight) with the highest weight from Frontier;
  end
end

Algorithm 2. Bidirectional Beam Search

Input: The set of traces C, Beam width B, Length of sequence L, Length of k-mer k, Target k-mer v (in forward direction) and v' (in backward direction).
Output: The best length-L path in the De Bruijn graph.

Function *BidirectionalBeamSearch*(C, B, L, k, v, v')
  /* Forward beam search */
  Learn $\mathcal{D}_C$ from the set of traces C;
  $\text{path}_f, \text{weight}_f \leftarrow \textit{BeamSearch}(\mathcal{D}_C, B, L, k, v)$;
  /* Reverse beam search */
  $C'\leftarrow C$ with every trace reversed;
  Learn $\mathcal{D}_{C'}$ from the set of traces C';
  $\text{path}_b, \text{weight}_b \leftarrow \textit{BeamSearch}(\mathcal{D}_{C'}, B, L, k, v')$
  end
/* Select the best path */
if $\textit{path}_f.\textit{len}() \neq \textit{path}_b.\textit{len}()$ **then**
  return the longer path of path_f and path_b;
else
  if $\textit{weight}_f > \textit{weight}_b$ **then**
    return path_f;
  else
    return path_b;
  end
end

B and the order of the Markov chain k. Due to the fact that BBS relies on (k+1)-mer profile for trace reconstruction, the algorithm would inevitably fail if one of the (k+1)-mer in the encoded sequence appears erroneous in all the traces. As a result, a careful choice of k is needed for datasets with high error rates and low coverage. In addition, the optimal choice of B can also vary with different coverages. Choosing the optimal values can again enhance the accuracy and efficiency of the BBS algorithm (Figures S2 and S4).

It is also worth investigating whether the k-MC model can be used in conjunction with error correction codes (ECC).[33,34] Though not tested in this work, ECC can be incorporated smoothly into the beam search procedure, where paths that do not satisfy the ECC requirements are filtered out before pushing into the frontier.

RESOURCE AVAILABILITY

Lead contact

Requests for further information and resources should be directed to and will be fulfilled by the lead contact, Niranjan Nagarajan (nagarajann@a-star.edu.sg).

Materials availability

This study did not generate any new materials.

Data and code availability

- The post-processed datasets used for the experiments from Bar-Lev et al.,[32] Srinivasavaradhan et al.,[19] and Chandak et al.[6] have been deposited at Zenodo under https://doi.org/10.1101/2025.04.16.644694 and are publicly available as of the date of publication.
- The implementation of BBS is available in GitHub at https://github.com/GZHoffie/bbs, and the scripts for reproducibility are available at https://github.com/GZHoffie/bbs-test.
- Any additional information required to reanalyze the data reported in this paper is available from the lead contact upon request.

ACKNOWLEDGMENTS

This research was supported by National Research Foundation Investigatorship grant (NRFI09-0015) to N.N.; STI2030-Major Projects 2022ZD0212400, STCSM grant 24510714300 and 20DZ2254400, GuangDong Basic and Applied Basic Research Foundation grant 2023B1515120006, and SJTU Science-Medicine interdisciplinary grant 24X010301456 to H.X.; Z.G., P.S., and G.Y.G. were supported by the National University of Singapore Research Scholarship.

AUTHOR CONTRIBUTIONS

Conceptualization, Z.G. and H.X.; methodology, Z.G. and P.S.; software, Z.G.; data curation, Z.G., P.S., and G.Y.G.; investigation, Z.G. and H.X.; writing – original draft, Z.G.; writing – review and editing, Z.G., H.X., P.S., L.W., and N.N.; supervision, L.W. and N.N.

DECLARATION OF INTERESTS

The authors declare no competing interests.

STAR★METHODS

Detailed methods are provided in the online version of this paper and include the following:

- KEY RESOURCES TABLE
- METHOD DETAILS
  - Problem formulation
  - Bidirectional beam search (BBS) algorithm
- QUANTIFICATION AND STATISTICAL ANALYSIS
- ADDITIONAL RESOURCES

SUPPLEMENTAL INFORMATION

Supplemental information can be found online at https://doi.org/10.1016/j.isci.2025.113791.

Received: June 27, 2025
Revised: August 28, 2025
Accepted: October 14, 2025
Published: October 21, 2025

REFERENCES

1. Bornholt, J., Lopez, R., Carmean, D.M., Ceze, L., Seelig, G., and Strauss, K. (2016). A dna-based archival storage system. In Proceedings of the twenty-first international conference on architectural support for programming languages and operating systems, pp. 637–649.
2. Doricchi, A., Platnich, C.M., Gimpel, A., Horn, F., Earle, M., Lanzavecchia, G., Cortajarena, A.L., Liz-Marzán, L.M., Liu, N., Heckel, R., et al. (2022). Emerging approaches to dna data storage: challenges and prospects. ACS Nano *16*, 17552–17571.
3. Yu, M., Tang, X., Li, Z., Wang, W., Wang, S., Li, M., Yu, Q., Xie, S., Zuo, X., and Chen, C. (2024). High-throughput dna synthesis for data storage. Chem. Soc. Rev. *53*, 4463–4489.
4. Shomorony, I., and Heckel, R. (2022). Information-theoretic foundations of dna data storage. Found. Trends Commun. Inf. Theory *19*, 1–106.
5. Levenshtein, V.I. (1997). Reconstruction of objects from the minimum number of distorted patterns. Dokl. Akad. Nauk *354*, 593–596. Russian Academy of Sciences.
6. Chandak, S., Neu, J., Tatwawadi, K., Mardia, J., Lau, B., Kubit, M., Hulett, R., Griffin, P., Wootters, M., Weissman, T., et al. (2020). Overcoming high nanopore basecaller error rates for dna storage via basecaller-decoder integration and convolutional codes. In ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (IEEE), pp. 8822–8826.
7. Wang, Y., Zhao, Y., Bollas, A., Wang, Y., and Au, K.F. (2021). Nanopore sequencing technology, bioinformatics and applications. Nat. Biotechnol. *39*, 1348–1365.
8. McNally, B., Singer, A., Yu, Z., Sun, Y., Weng, Z., and Meller, A. (2010). Optical recognition of converted dna nucleotides for single-molecule dna sequencing using nanopore arrays. Nano Lett. *10*, 2237–2244.
9. Illumina, I.. Novaseq 6000 system specifications. https://sapac.illumina.com/systems/sequencing-platforms/novaseq/specifications.html(a.
10. Illumina, I.. Run time estimates for each sequencing step on illumina sequencing platforms. https://knowledge.illumina.com/instrumentation/general/instrumentation-general-reference_material-list/000001540(b.
11. Sokolovskii, R., Ramirez Garcia, R., and Heinis, T. (2024). Adaptive sampling in nanopore sequencing for pcr-free random access in dna data storage. Preprint at bioRxiv. https://doi.org/10.1101/2024.11.05.622081.
12. Delahaye, C., and Nicolas, J. (2021). Sequencing dna with nanopores: Troubles and biases. PLoS One *16*, e0257521.
13. Antkowiak, P.L., Lietard, J., Darestani, M.Z., Somoza, M.M., Stark, W.J., Heckel, R., and Grass, R.N. (2020). Low cost dna data storage using photolithographic synthesis and advanced information reconstruction and error correction. Nat. Commun. *11*, 5345.
14. Rashtchian, C., Makarychev, K., Racz, M., Ang, S., Jevdjic, D., Yekhanin, S., Ceze, L., and Strauss, K. (2017). Clustering billions of reads for dna data storage. Adv. Neural Inf. Process. Syst. *30*.
15. Edgar, R.C. (2004). Muscle: multiple sequence alignment with high accuracy and high throughput. Nucleic Acids Res. *32*, 1792–1797.
16. Katoh, K., Misawa, K., Kuma, K.-i., and Miyata, T. (2002). Mafft: a novel method for rapid multiple sequence alignment based on fast fourier transform. Nucleic Acids Res. *30*, 3059–3066.
17. Xie, R., Zan, X., Chu, L., Su, Y., Xu, P., and Liu, W. (2023). Study of the error correction capability of multiple sequence alignment algorithm (mafft) in dna storage. BMC Bioinf. *24*, 111.
18. Pagès-Gallego, M., and de Ridder, J. (2023). Comprehensive benchmark and architectural analysis of deep learning models for nanopore sequencing basecalling. Genome Biol. *24*, 71.
19. Srinivasavaradhan, S.R., Gopi, S., Pfister, H.D., and Yekhanin, S. (2021). Trellis bma: Coded trace reconstruction on ids channels for dna storage. In 2021 IEEE International Symposium on Information Theory (ISIT) (IEEE), pp. 2453–2458.
20. Bhardwaj, V., Pevzner, P.A., Rashtchian, C., and Safonova, Y. (2021). Trace reconstruction problems in computational biology. IEEE Trans. Inf. Theory *67*, 3295–3314.
21. Batu, T., Kannan, S., Khanna, S., and McGregor, A. (2004). Reconstructing strings from random traces. SODA *4*, 910–918.
22. Gopalan, P.S., Yekhanin, S., Ang, S.D., Jojic, N., Racz, M., Strauss, K., and Ceze, L. (2018). Trace reconstruction from noisy polynucleotide sequencer reads. US Patent App. 15/536,115.
23. Lin, D., Tabatabaee, Y., Pote, Y., and Jevdjic, D. (2022). Managing reliability skew in dna storage. In Proceedings of the 49th Annual International Symposium on Computer Architecture, pp. 482–494.
24. Organick, L., Ang, S.D., Chen, Y.-J., Lopez, R., Yekhanin, S., Makarychev, K., Racz, M.Z., Kamath, G., Gopalan, P., Nguyen, B., et al. (2018). Random access in large-scale dna data storage. Nat. Biotechnol. *36*, 242–248.
25. Ma, X., Shao, Y., Tian, L., Flasch, D.A., Mulder, H.L., Edmonson, M.N., Liu, Y., Chen, X., Newman, S., Nakitandwe, J., et al. (2019). Analysis of error profiles in deep next-generation sequencing data. Genome Biol. *20*, 50.
26. Song, L., Geng, F., Gong, Z.-Y., Chen, X., Tang, J., Gong, C., Zhou, L., Xia, R., Han, M.-Z., Xu, J.-Y., et al. (2022). Robust data storage in dna by de bruijn graph-based de novo strand assembly. Nat. Commun. *13*, 5361.
27. Sabary, O., Yucovich, A., Shapira, G., and Yaakobi, E. (2024). Reconstruction algorithms for dna-storage systems. Sci. Rep. *14*, 1951.
28. Qin, Y., Zhu, F., Xi, B., and Song, L. (2024). Robust multi-read reconstruction from noisy clusters using deep neural network for dna storage. Comput. Struct. Biotechnol. J. *23*, 1076–1087.
29. Zheng, X., Xie, R., Yao, X., Su, Y., Chu, L., Xu, P., and Liu, W. (2024). A generative adversarial network for multiple reads reconstruction in dna storage. Sci. Rep. *14*, 32071.
30. Nahum, Y., Ben-Tolila, E., and Anavy, L. (2021). Single-read reconstruction for dna data storage using transformers. Preprint at arXiv. https://doi.org/10.48550/arXiv.2109.05478.
31. Edgar, R.C. (2022). Muscle5: High-accuracy alignment ensembles enable unbiased assessments of sequence homology and phylogeny. Nat. Commun. *13*, 6968.
32. Bar-Lev, D., Orr, I., Sabary, O., Etzion, T., and Yaakobi, E. (2025). Scalable and robust dna-based storage via coding theory and deep learning. Nat. Mach. Intell. *7*, 639–649.

33. Blawat, M., Gaedke, K., Hütter, I., Chen, X.-M., Turczyk, B., Inverso, S., Pruitt, B.W., and Church, G.M. (2016). Forward error correction for dna data storage. Procedia Comput. Sci. *80*, 1011–1022.

34. Sima, J., Raviv, N., Schwartz, M., and Bruck, J. (2023). Error correction for dna storage. IEEE BITS Inform. Theory Mag. *3*, 78–94.

35. Chase, Z. (2021). New lower bounds for trace reconstruction. Preprint at arXiv. https://doi.org/10.48550/arXiv.1905.03031.

36. Cheng, K., Grigorescu, E., Li, X., Sudan, M., and Zhu, M. (2024). On *k*-mer-based and maximum likelihood estimation algorithms for trace reconstruction. In 2024 IEEE International Symposium on Information Theory (ISIT) (IEEE), pp. 879–884.

37. De, A., O'Donnell, R., and Servedio, R.A. (2017). Optimal mean-based algorithms for trace reconstruction. In Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, pp. 1047–1056.

38. Holenstein, T., Mitzenmacher, M., Panigrahy, R., and Wieder, U. (2008). Trace reconstruction with constant deletion probability and related results. In Proceedings of the nineteenth annual ACM-SIAM symposium on Discrete algorithms, pp. 389–398.

39. Levenshtein, V.I. (2001). Efficient reconstruction of sequences from their subsequences or supersequences. J. Combin. Theor. *93*, 310–332.

40. Viswanathan, K., and Swaminathan, R. (2008). Improved string reconstruction over insertion-deletion channels. In Proceedings of the nineteenth annual ACM-SIAM symposium on Discrete algorithms, pp. 399–408.

41. Viterbi, A. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. IEEE Trans. Inf. Theory *13*, 260–269.

42. Hoare, C.A.R. (1961). Algorithm 65: find. Commun. ACM *4*, 321–322.

STAR★METHODS

KEY RESOURCES TABLE

REAGENT or RESOURCE	SOURCE	IDENTIFIER
Deposited data		
Microsoft clustered nanopore reads (CNR) dataset	Srinivasavaradhan et al.[19]	https://github.com/microsoft/clustered-nanopore-reads-dataset
DNAformer dataset	Bar-Lev et al.[32]	https://zenodo.org/records/13896773
Nanopore DNA storage basecaller data	Chandak et al.[6]	https://github.com/shubhamchandak94/nanopore_dna_storage_ data
Software and algorithms		
BBS	This work	https://github.com/GZHoffie/bbs
MUSCLE	Edgar et al.[31]	https://www.drive5.com/muscle/
Trellis BMA	Srinivasavaradhan et al.[19]	https://github.com/microsoft/TrellisBMA
ITR	Sabary et al.[27]	https://github.com/omersabary/Reconstruction
CPL	Bar-Lev et al.[32]	https://github.com/itaiorr/Deep-DNA-based-storage

METHOD DETAILS

Problem formulation

Let us denote the set of alphabets to be $\Sigma = \{A,C,G,T\}$. Algorithms for trace reconstruction problems take a set of N traces, $C = \{c_1, \ldots, c_N\}$, as input, where each c_i in C is a string Σ^* that is some erroneous version of a seed string $s \in \Sigma^L$, and aim to output the seed string $\hat{s}$ such that $s = \hat{s}$ with high probability. For simplicity, we denote $S_{i:j}$ as the subsequence of S from the i-th to the j-th character, $S_i, S_{i+1}, \ldots, S_j$. We also write $S_{i:j} = S'_{i:j}$ to represent the event $S_i = S'_i \wedge S_{i+1} = S'_{i+1} \wedge \cdots \wedge S_j = S'_j$.

While the intuition and the purpose of the trace reconstruction problem are clear, the formal definition of the optimization objective varies across different methods. MSA-based methods attempt to find the consensus in the alignment of all the traces that maximize the global alignment score,[13,15–17] while deep-learning-based methods minimize the loss function that captures the reconstruction errors measured using edit distance.[26,29] The most widely used definition is to model the synthesis and sequencing process as an insertion-deletion-substitution (IDS) channel and attempt to find the seed string that maximizes the likelihood of observing the traces.[19,20]

IDS-channel-based trace reconstruction

Input: A set of traces C.
Assumptions.
(1) Each trace $c_i \in C$ is independent.
(2) Each trace c_i is a modified version of the seed string $s \in \Sigma^L$, where insertion, deletion, substitution, and matching happen at each position with probability p_I, p_D, p_S, p_M respectively.
Output: $\arg\max_{\hat{s} \in \Sigma^L} \Pr[C|\hat{s}]$.

Under the assumptions, the log likelihood of observing the traces can be calculated by

$$\log\Pr[C|\hat{s}] = \sum_{c_i \in C} \log\Pr[c_i|\hat{s}] = \sum_{c_i \in C} (M_i \cdot \log p_M + S_i \cdot \log p_S + D_i \cdot \log p_D + I_i \cdot \log p_I) \quad \text{(Equation 1)}$$

where M_i, S_i, D_i, and I_i denote the total number of matches, substitutions, deletions, and insertions needed to transform $\hat{s}$ to the trace c_i. The goal can then be interpreted as finding the length-L string $\hat{s}$ with the highest alignment score defined by p_I, p_D, p_S, and p_M to all the traces.

This problem definition, though supported by many theoretical works,[35–40] might encounter problems when applied to real-life data. Firstly, solving the optimization is hard. In the general case, the best existing solution to the optimization problem is brute-force,[20] taking exponential time with respect to sequence length L. Secondly, even if the optimal string $\hat{s}$ is found, it doesn't necessarily reflect the true alignment. In real datasets, the fixed p_I, p_D, p_S, and p_M can vary across different clusters and positions within the traces. Notably, error rates tend to be higher at the ends of reads,[13] and vary with homopolymer length as well as GC-content,[12]

leading to inconsistencies and reducing the reliability of alignment scores. Choosing the correct values for p_I,p_D,p_S, and p_M is challenging since different definitions of these parameters can result in drastically different alignments and consensus sequences. Furthermore, when incorporating prior knowledge of the encoded sequence length, the optimal alignment may not accurately represent the true consensus, as illustrated in the example in Figure 1.

Therefore, we propose an alternative objective. Instead of finding the seed string that maximizes the likelihood of observing the traces, we model the traces as observations of a k-th order Markov chain (k-MC) consisting of L variables, S_1, …,S_L, where $S_i \in \Sigma = \{A,C,G,T\}$, and assume that for all $i = k+1,k+2, \ldots,L$,

$$\Pr[S_i|S_{1:i-1}] = \Pr[S_i|S_{i-k:i-1}],$$

and try to predict the most probable trace that is generated by the k-MC. We can then learn the conditional probabilities $\mathcal{D}_C$ from the set of traces C, and perform the maximum likelihood estimate (MLE) of the future trace.

K-MC-based trace reconstruction

Input: A set of traces C.
Assumptions: Each trace $c_i \in C$ can be viewed as independent observations of a k-th order Markov chain.
Output: $\text{argmax}_{\hat{s} \in \Sigma^L} \Pr_{S \sim \mathcal{D}_C}[S = \hat{s}]$.

Under the k-MC assumption, the joint probability of observing $\hat{s}$ as the next observation can be calculated by

$$\log \Pr_{S \sim \mathcal{D}_C}[S = \hat{s}] = \log \Pr_{S \sim \mathcal{D}_C}[S_{1:k+1} = \hat{s}_{1:k+1}] + \sum_{i=k+2}^{L} \log \Pr_{S \sim \mathcal{D}_C}[S_i = \hat{s}_i|S_{i-k:i-1} = \hat{s}_{i-k:i-1}]. \quad \text{(Equation 2)}$$

Intuitively, the new problem definition also tries to find the underlying seed string used to generate the traces. Under the IDS-channel assumption, if $p_M = max\{p_M,p_I,p_D,p_S\}$, the most probable future trace given a seed string s is s itself, with the highest likelihood of p_M^L.

The advantages of this problem definition lie in the following: Firstly, exact as well as heuristic algorithms to find the most probable observation in a Markov model, such as the Viterbi algorithm, are well studied.[41] Secondly, it uses weaker assumptions than the previous definition based on the IDS-channel, which is essentially a first-order hidden Markov model with fixed probability values for errors. Since k is chosen such that all the (k+1)-mers are unique in the sequences, the conditional probabilities $\mathcal{D}_C$ for each cluster C allow us to learn different error probabilities for each position in each cluster. Thirdly, calculating the likelihood for the next trace doesn't require any alignment or calculation of edit distance among the traces, leading to high efficiency. Finally, the value $\log \Pr_{S \sim \mathcal{D}_C}[S = \hat{s}]$ represents the joint likelihood of observing $\hat{s}$ as the next trace, allowing us to assign a confidence score to the output, as well as compare the goodness of the output sequence, aiding future post-processing and the downstream decoding tasks.

Next, we propose an algorithm, bidirectional beam search (BBS), to efficiently find the most probable observation of the estimated k-MC with high probability.

Bidirectional beam search (BBS) algorithm

The main idea of the bidirectional beam search algorithm is to incorporate the learned parameters k-MC model into the de Bruijn graph and find the best path that represents the most probable sequence generated by the k-MC model using beam search. The beam search is done in two directions, one on the original traces and one on the reversed traces, for optimal performance. An illustration of the BBS algorithm is shown in Figure 2.

The first step of the BBS algorithm is to choose the value of k. In our algorithm, we choose the smallest k within a predefined range [k_{min}, k_{max}] such that all the k-mers are unique within each trace. The expected value of k is $O(logL)$ (Figure S3). In rare cases where such k cannot be found, k_{max} is used. Such cases can be avoided by carefully designing the encoding scheme in practice. The uniqueness ensures that the de Bruijn graph built using the (k+1)-mers in the traces is a directed acyclic graph (DAG). We also choose the smallest k that satisfies uniqueness, as a smaller k can lead to better predictions of the conditional probabilities and, consequently, a better reconstruction performance. We then store the (k+1)-mer counts in a hash table. This process takes time linear to the size of the input.

We can now build a modified de Bruijn graph with the following specifications,

(1) **ices** (denoted V): All possible (k+1)-mers that appear in the traces, plus a special vertex `Start` indicating the start of the reads.
(2) **Edges** (denoted E): For each (k+1)-mers $v \in \Sigma^{k+1}$ that have appeared at the start of each trace, a directed edge ($Start$,v) is added. We also add a directed edge between pairs of (k+1)-mers (u,v) if the last k bases of u match the first k bases of v. Note that this does not require adjacency of u and v in the same sequence.
(3) **Edge weights**: For each edge ($Start$,v)$\in E$, we assign the weight to be the estimated joint probability of the first (k+1)-mer in s being v,

$$w((Start, v)) = \log \widehat{\Pr}_{S\sim\mathcal{D}_C}[s_{1:k+1} = v_{1:k+1}]$$

while for other edges $(u,v)\in E$, we assign the weight to be the estimated conditional probability.

$$w((u,v)) = \log \widehat{\Pr}_{S\sim\mathcal{D}_C}[s_i = v_{k+1} | s_{i-k:i-1} = v_{1:k}].$$

Based on the k-MC assumption, we can estimate the joint probability and the conditional probabilities using the maximum likelihood estimate,

$$\widehat{\Pr}_{S\sim\mathcal{D}_C}[s_1 = v_1, \ldots, s_{k+1} = v_{k+1}] = \frac{n_1(v)}{N},$$

where $n_1(v)$ refers to the number of times the $(k+1)$-mer v appears as the first $(k+1)$-mer in the traces, while

$$\widehat{\Pr}_{S\sim\mathcal{D}_C}[s_i = v_{k+1} | s_{i-k:i-1} = v_{1:k}] = \frac{n((v_1, v_2, \ldots, v_k, v_{k+1}))}{\sum_{j\in\Sigma} n((v_1, v_2, \ldots, v_k, j))},$$

where $n(v) = n((v_1, \ldots, v_{k+1}))$ refers to the number of times $(k+1)$-mer v appears anywhere in the traces. In practice, especially with a high error rate and small coverage, the numbers $n(v)$ can be small, resulting in inaccurate estimations. We therefore apply Laplace smoothing,

$$\widehat{\Pr}_{S\sim\mathcal{D}_C}[s_i = v_{k+1} | s_{i-k:i-1} = v_{1:k}] = \frac{n((v_1, v_2, \ldots, v_k, v_{k+1}))+\alpha}{\sum_{j\in\Sigma}[n((v_1, v_2, \ldots, v_k, j))+\alpha]}, \quad \text{(Equation 3)}$$

and set the default $\alpha = 1$. Intuitively, a larger α would lead to larger penalties on "rare" paths that very few traces agree.

The construction is similar to the typical de Bruijn graphs used for assembly, except for the assignment of edge weights. With this construction, a path starting from `Start` vertex and containing L-k+1 vertices would correspond to a string $s\in\Sigma^L$, and the weight of the edges in the path sums up to $\log \Pr_{S\sim\mathcal{D}_C}[S = s]$. Thus, the trace reconstruction problem is reduced to a fixed-length longest path problem,

Fixed-length longest path problem

Input: A directed acyclic graph $G(V,E)$, the starting vertex $u\in V$ and an ending vertex $v\in V$, and a predefined length l.
Output: A length l path that starts from u, ends in v, with the maximum sum of edge weights.

In our constructed graph, the length $l = L$-k+1, and the goal vertex v are set heuristically to be the most frequently appearing $(k+1)$-mer at the end of the traces.

A brute-force algorithm to solve the fixed-length longest path problem is to perform l iterations of breadth-first search (BFS) that allows repeated visiting of the same vertex. However, such an algorithm may result in $O(|\Sigma|^l)$ possible paths of length l in the end. Therefore, we opt to use beam search to approximate the optimal length-l path in the constructed graph. The benefits of using beam search involve its fast running time and its ability to output multiple candidate good consensus sequences, allowing more room for errors. By running exactly l iterations of beam search, we also leverage the prior information of the sequence length. A detailed description of the beam search can be found in Algorithm 1.

In each iteration of beam search, at most B paths are kept in the frontier, and at most $B\cdot|\Sigma|$ paths are explored. The calculation of conditional probabilities is done during the beam search using Equation 3, which costs $O(1)$ time for each calculation as the $(k+1)$-mer profile is stored using hash tables. At the end of each iteration, the top B paths with the highest weight are inserted into the new frontier. The top B tuples are selected using Hoare's selection algorithm,[42] costing $O(B\cdot|\Sigma|)$ time. This bounds the time complexity of the beam search to be $O(LB|\Sigma|)$, which can be regarded as $O(L)$ for small B and $|\Sigma|$.

Notice that the joint probability in Equation 2 can also be calculated in a reversed manner,

$$\log \Pr_{S\sim\mathcal{D}_C}[S = \hat{s}] = \log \Pr_{S\sim\mathcal{D}_C}[S_{L-k:L} = \hat{s}_{L-k:L}] + \sum_{i=1}^{L-k-1} \log \Pr_{S\sim\mathcal{D}_C}[S_i = \hat{s}_i | S_{i+1:i+k} = \hat{s}_{i+1:i+k}], \quad \text{(Equation 4)}$$

which hints that the beam search can also be done in the other direction, which starts from the end of the traces. An intuitive way is to perform a beam search in both directions and select the best sequence with the highest path weight, as shown in Algorithm 2.

In the actual implementation, the conditional probabilities $\mathcal{D}_{C'}$ are calculated from the same $(k+1)$-mer profile instead of reversing every trace in C to save time. As Algorithm 2 essentially performs Algorithm 1 twice, the time complexity is also $O(L)$.

QUANTIFICATION AND STATISTICAL ANALYSIS

Similar to previous works, we use three metrics to measure the correctness of the algorithms, including the following,

(1) Success rate, defined by the percentage of the clusters that are reconstructed correctly without any error.
(2) Average Hamming distance per cluster, where the Hamming distance between the ground truth sequence s and the predicted sequence $\widehat{s}$ is

$$d_H = \sum_{i=1}^{L} I(\text{length}(\widehat{s}) < i \vee s_i \neq \widehat{s}_i)$$

where $I(A)$ is the indicator function which returns 1 if A is evaluated to `true` and 0 otherwise.

(3) Average edit distance per cluster, where the edit distance is calculated from the optimal alignment between s and $\widehat{s}$.

ADDITIONAL RESOURCES

Scripts for reproducing the experiments in the paper can be found at: https://github.com/GZHoffie/bbs-test.